SYSTEM S1 — SOFTWARE DESIGN DOCUMENT

Single sign-on, JWT issuance & verification, RBAC, and the platform-wide audit trail — the security backbone every other system calls before processing a request

CONSOLIDATES FORMER MODULES

M1 Identity & Access

DOCUMENT INFORMATION

Document type	Software Design Document — Developer Edition
Intended audience	The developer (or team) building System 1 — Identity & Access. Read S0 first.
Version	1.0
Date	2026-06-15
Companion documents	S0 Shared Platform Reference · S2 Workforce & Payroll · S3 Hospitality Operations · S4 Finance & BI

Table of Contents

> **NOTE**
>
> This document is the complete build specification for **System 1 — Identity & Access**, one Laravel 11 application backed by one MySQL database (`wh_s1_db`). S1 consolidates the former M1 microservice **unchanged in scope** — it remains the platform's single source of truth for authentication, authorization, and the audit trail.
>
> Before reading further, make sure you have read **S0 — Cross-System Integration & Platform Conventions**, which defines the gateway, the JWT flow, the cross-system event bus, the permission-naming convention (ID9), and the platform conventions (D1–D14) referenced throughout this document by ID.

1. Purpose, Scope & Responsibilities

1.1 Why S1 exists

Every HTTP request that reaches S2, S3, or S4 must carry a JWT issued and signed by S1. **No other system contains authentication logic, stores a password, or issues a token.** S1 is the only system that:

- Owns the **users**, **roles**, **permissions**, **role_permissions**, and **user_roles** tables for the *entire* ERP — even though S2–S4 each define the permission strings relevant to their own domain (Section 6.4), the seed data for all of them lives in S1's database.
- Issues RS256-signed JWT **access tokens** (60-minute TTL) and long-lived, rotating **refresh tokens** (30-day TTL, SHA-256 hashed at rest).
- Exposes `POST /api/v1/auth/verify` — the single endpoint S2, S3, and S4 call on every authenticated request (S0 §4.1, §4.3).
- Exposes `GET /api/v1/auth/jwks` for signing-key rotation, entirely internal to S1 (S0 §4.4).
- Enforces password policy (minimum length, complexity, expiry, history) and account lockout after repeated failed logins.
- Maintains the **append-only** `audit_logs` **table** — the platform's security audit trail (also used for the quarterly DR restore-drill log, D14).
- Reacts to employee lifecycle events from S2 to automatically **provision, sync, and deactivate** linked user accounts (Decisions ID1, ID2).

1.2 What is explicitly out of scope for S1

Out of scope	Where it actually lives
Employee/HR records (name, department, position, pension category)	S2 — S1 only stores a `display_name` copy and an `employee_id` reference (ID2)
Business permission *logic* for S2/S3/S4 domains (e.g. what `S2.payroll.runs.execute` actually gates)	Defined in each system's own SDD; S1 only stores the permission *string* and its role grants
Any UI	Out of scope for every system SDD
Tracing / APM	Deferred platform-wide (D10)

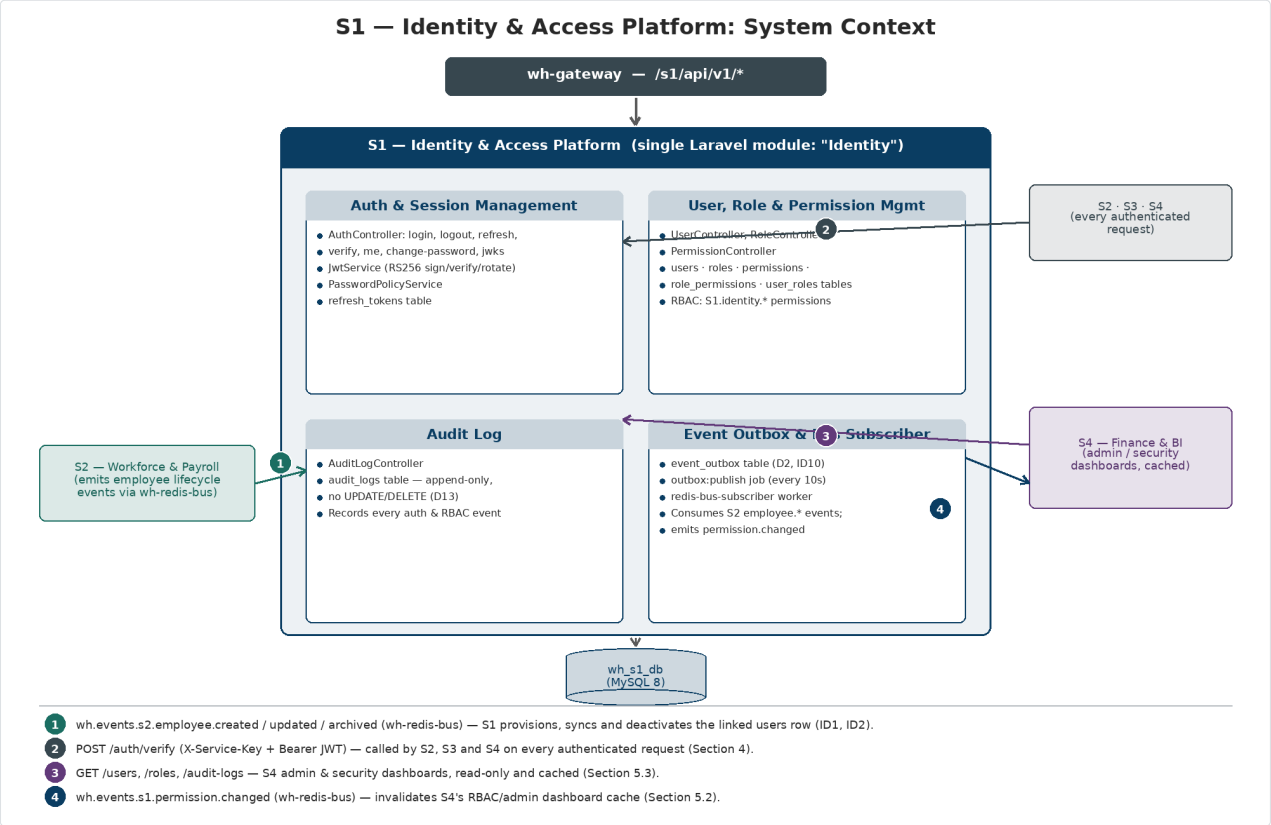
1.3 Relationship to the original M1 design

S1 is a **direct, unchanged carry-forward of M1** — there is no internal split, no merge with another module (S0 §6: *"Single module, no internal split needed"*). Every table, controller, and business rule in the original `M1_Identity_Access_Service.md` and `APPENDIX_A1_Permission_Catalog_M1-M3.md` (M1 section) carries forward. This document differs from those sources in exactly three ways, all driven by the four-system consolidation:

Change	Source	Detail
`users` table gains `display_name`	ID2	Section 3.1, Section 5.2
New `event_outbox` table + outbox worker	D2 / ID10	Section 3.8, Section 8
Permission strings renamed from `M1.*` to `S1.identity.*`; user-management permission grants re-derived from the System Access Matrix (S0 §7.2)	ID9	Section 6

2. Internal Architecture

S1 is a single Laravel 11 application — **one module, internally called `Identity`** — with no sub-module boundaries to manage. The diagram below shows S1's four functional areas and every interaction that crosses its system boundary. Anything not shown here does not happen; S1's cross-system surface is exhaustively listed in S0 Section 5.



S1's four internal functional areas (Auth & Session, User/Role/Permission management, the append-only audit log, and the event outbox/bus subscriber) plus every external caller and callee. Numbered callouts are explained in the legend below the diagram.

2.1 Functional areas

Area	Responsibility	Key classes (Laravel conventions)
Auth & Session Management	Login, logout, refresh, verify, jwks, change-password	`AuthController`, `JwtService`, `PasswordPolicyService`, `refresh_tokens` table
User, Role & Permission Management	CRUD for users/roles/permissions, role assignment, permission sync	`UserController`, `RoleController`, `PermissionController`, `users` / `roles` / `permissions` / `role_permissions` / `user_roles` tables
Audit Log	Append-only record of every auth & RBAC event	`AuditLogController`, `AuditService`, `audit_logs` table
Event Outbox & Bus Subscriber		

Area	Responsibility	Key classes (Laravel conventions)
	Reliable cross-system event publishing and consumption	`event_outbox` table, `outbox:publish` job, `redis-bus-subscriber` Horizon worker

2.2 A subtlety: S1 does not call its own `/auth/verify`

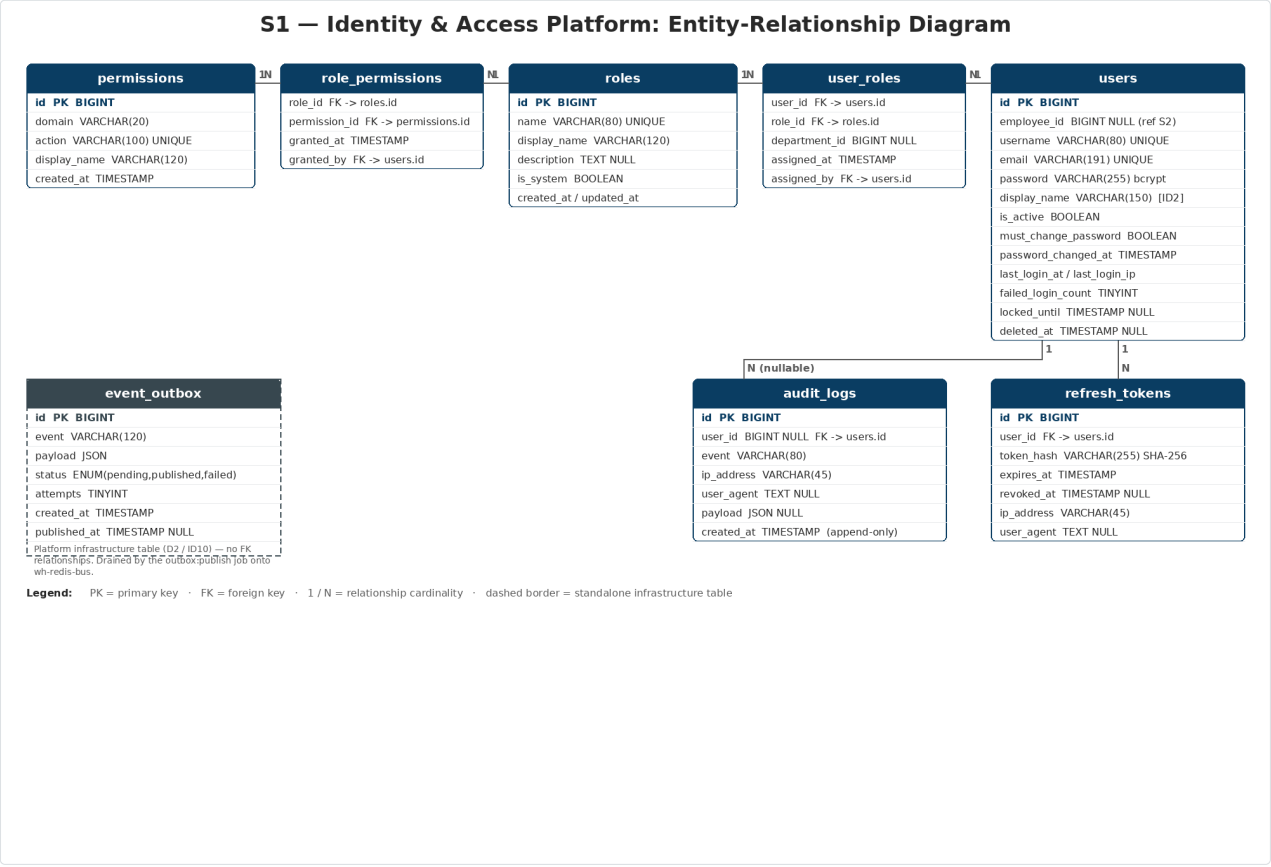
S2, S3, and S4 each run a `JwtAuthenticate` middleware that calls `POST {S1_URL}/api/v1/auth/verify` (S0 §4.1, step 4). **S1's own protected endpoints do not do this** — S1 holds its own RS256 key pair, so its local `JwtAuthenticate` middleware decodes and validates the JWT signature *directly, in-process*. The `/auth/verify` endpoint exists purely as a service exposed *to* S2–S4; it is never called by S1 itself.

2.3 Folder structure

```
s1-identity-access/
app/Http/Controllers/
  AuthController.php        # login, logout, refresh, verify, jwks, me, changePassword
  UserController.php        # CRUD, deactivate, forceLogout, resetPassword, assignRole,
removeRole
  RoleController.php         # CRUD, syncPermissions
  PermissionController.php   # index, byDomain
  AuditLogController.php     # index, forUser
app/Http/Middleware/
  JwtAuthenticate.php         # LOCAL decode of RS256 JWT (S1's own protected routes)
  CheckPermission.php         # checks decoded payload for named S1.identity.* permission
  ThrottleFailedLogins.php     # rate-limits login attempts per IP (gateway also rate-
limits, S0 §3)
  ServiceKeyAuthenticate.php   # guards POST /auth/verify (S0 §4.3)
app/Http/Requests/
  LoginRequest.php  CreateUserRequest.php  AssignRoleRequest.php
app/Models/
  User.php  Role.php  Permission.php  UserRole.php  RolePermission.php
  RefreshToken.php  AuditLog.php  EventOutbox.php
app/Services/
  JwtService.php             # issue / decode / rotate tokens (RS256), JWKS publication
  PasswordPolicyService.php   # validate complexity + history
  AuditService.php             # static log() helper
  OutboxService.php            # write event_outbox rows transactionally (D2)
app/Events/
  UserLoggedIn.php  UserLoggedOut.php  PermissionChanged.php
  UserDeactivated.php  PasswordReset.php
app/Listeners/
  WriteAuditLog.php
  CreateLinkedUserAccount.php    # consumes wh.events.s2.employee.created (ID2)
  SyncLinkedUserAccount.php       # consumes wh.events.s2.employee.updated (ID2)
  DeactivateLinkedUserAccount.php # consumes wh.events.s2.employee.archived (ID1)
app/Jobs/
  PublishOutboxEvents.php   # outbox:publish, every 10s (D2)
database/migrations/        # 8 migration files (Section 3)
routes/api.php
config/ jwt.php  password_policy.php  services.php
Dockerfile docker-compose.yml .env
```

3. Data Model

S1's database, `wh_s1_db`, contains **8 tables**: the 7 original M1 tables plus the new `event_outbox` table (D2 / ID10). The ER diagram below shows all 8 tables with their columns and relationships.



S1's complete data model: the RBAC chain (permissions ↔ role_permissions ↔ roles ↔ user_roles ↔ users), the users table's session/security columns including the new display_name (ID2), the per-user refresh_tokens and audit_logs tables, and the standalone event_outbox infrastructure table.

3.1 Table: `users`

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
employee_id	BIGINT UNSIGNED NULL	Cross-system reference to S2's `employees.id` — **not a foreign key** (different database); populated/ maintained via the event listeners in Section 8
username	VARCHAR(80) UNIQUE	Login handle, e.g. `abel.tadesse`
email	VARCHAR(191) UNIQUE	
password	VARCHAR(255)	Bcrypt hash — never stored or logged in plaintext
display_name	**VARCHAR(150)**	**New — ID2.** Copied from S2's `employee.full_name` on `employee.created` /

Column	Type / Constraint	Notes
		`employee.updated`. Source of the JWT `name` claim (S0 §4.2)
is_active	TINYINT(1) DEFAULT 1	`0` = deactivated, cannot log in (D13 soft-delete flag)
must_change_password	TINYINT(1) DEFAULT 0	Forces a password reset on next login; set `1` on provisioning
password_changed_at	TIMESTAMP NULL	
last_login_at	TIMESTAMP NULL	
last_login_ip	VARCHAR(45) NULL	IPv4/IPv6
failed_login_count	TINYINT UNSIGNED DEFAULT 0	Reset to `0` on a successful login
locked_until	TIMESTAMP NULL	Set when `failed_login_count` reaches the configured maximum (Section 5.4)
created_at / updated_at	TIMESTAMP	Laravel-managed
deleted_at	TIMESTAMP NULL	Soft delete (Laravel `SoftDeletes` trait, D13)

3.2 Table: `roles`

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
name	VARCHAR(80) UNIQUE	Slug, e.g. `hr_manager` — must match the 12 role slugs in S0 §7.2
display_name	VARCHAR(120)	Human label, e.g. “HR Manager”
description	TEXT NULL	
is_system	TINYINT(1) DEFAULT 0	System roles (Section 6.2) cannot be deleted; enforced in `RoleController::destroy`
created_at / updated_at	TIMESTAMP	

3.3 Table: `permissions`

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
domain	VARCHAR(20)	The `{domain}` segment of `S{n}.{domain}.{resource}.{action}` — e.g. `identity`, `hr`, `payroll`, `hotel` (was `module` in the original M1 schema, holding `M1` – `M9`)
action	VARCHAR(120) UNIQUE	Full permission string, e.g. `S1.identity.users.read`, `S2.payroll.runs.execute`

Column	Type / Constraint	Notes
display_name	VARCHAR(120)	Human-readable label for the admin UI
created_at	TIMESTAMP	Seeded at deployment from every system's permission catalog (Section 6.4)

> **NOTE**
>
> **This table is platform-wide, not S1-only.** It is seeded with the permission strings from *every* system's catalog (S1's own catalog in Section 6.1, plus the catalogs published in S2's, S3's, and S4's SDDs). S1 is simply where the table lives — the central RBAC authority. Seeding all four catalogs into `wh_s1_db` is part of S1's deployment runbook.

3.4 Table: `role_permissions`

Column	Type / Constraint	Notes
role_id	BIGINT UNSIGNED FK	→ `roles.id`
permission_id	BIGINT UNSIGNED FK	→ `permissions.id`
granted_at	TIMESTAMP	
granted_by	BIGINT UNSIGNED FK	→ `users.id` (the admin who granted it; NULL for seed-time grants)

Composite primary key `(role_id, permission_id)`.

3.5 Table: `user_roles`

Column	Type / Constraint	Notes
user_id	BIGINT UNSIGNED FK	→ `users.id`
role_id	BIGINT UNSIGNED FK	→ `roles.id`
department_id	BIGINT UNSIGNED NULL	Cross-system reference to S2's `departments.id` (not a FK); scopes the role to one department for roles like `department_head`
assigned_at	TIMESTAMP	
assigned_by	BIGINT UNSIGNED FK	→ `users.id`

Composite primary key `(user_id, role_id)`. A user may hold multiple roles (e.g. `hr_manager` + `report_viewer`).

3.6 Table: `refresh_tokens`

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
user_id	BIGINT UNSIGNED FK	→ `users.id`

Column	Type / Constraint	Notes
token_hash	VARCHAR(255)	SHA-256 hash of the raw token — the raw value is returned to the client once and never stored
expires_at	TIMESTAMP	30 days from issue
revoked_at	TIMESTAMP NULL	`NULL` = still valid. Set on rotation (`/auth/refresh`), logout, force-logout, or `employee.archived` (ID1)
ip_address	VARCHAR(45)	Issuing client IP
user_agent	TEXT NULL	Issuing client user agent
created_at	TIMESTAMP	

3.7 Table: `audit_logs`

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
user_id	BIGINT UNSIGNED NULL	`NULL` for anonymous/failed-login attempts
event	VARCHAR(80)	e.g. `login.success`, `login.failed`, `logout`, `permission.changed`, `user.deactivated`, `password.reset`
ip_address	VARCHAR(45)	
user_agent	TEXT NULL	
payload	JSON NULL	Contextual snapshot (e.g. which role/ permissions changed)
created_at	TIMESTAMP	

> **CONVENTION**
>
> **Append-only (D13).** No migration, controller, or admin tool may issue `UPDATE` or `DELETE` against `audit_logs`. This table is also the target of the quarterly DR-restore-drill log entry (D14) and the `super_admin` notification trail for outbox/subscriber failures (D4).

3.8 Table: `event_outbox` (new — D2, ID10)

Column	Type / Constraint	Notes
id	BIGINT UNSIGNED PK AI	
event	VARCHAR(120)	Channel name, e.g. `wh.events.s1.permission.changed`
payload	JSON	Full event envelope body (D3)
status	ENUM('pending','published','failed') DEFAULT 'pending'	

Column	Type / Constraint	Notes
attempts	TINYINT UNSIGNED DEFAULT 0	Incremented on each publish attempt
created_at	TIMESTAMP	
published_at	TIMESTAMP NULL	Set when `status` becomes `published`

Rows are written **in the same DB transaction** as the business change that causes them (e.g. `RoleController::syncPermissions`). The `outbox:publish` job (every 10s) reads `pending` rows, `PUBLISH`es them to `wh-redis-bus`, and marks them `published`; failures retry with backoff `[10s, 60s, 5m, 30m, 2h]` before `failed` (D2, D4). See Section 8.

---

4. API Endpoint Catalog

All endpoints are served under `/api/v1/` and reached externally via the gateway as `/s1/api/v1/...` (S0 §3). `Auth` column values: **Public** (no token), **JWT** (any valid access token, no specific permission), **JWT + perm** (valid token *and* the named permission in Section 6.1), **Service Key** (S0 §4.3 `ServiceKeyAuthenticate`).

Method	Endpoint	Description	Auth
GET	`/api/v1/health`	Health check — `{"status":"ok","system":"S1","version":"1.0"}`	Public
POST	`/api/v1/auth/login`	Authenticate with `{username, password}` ; issues access + refresh tokens	Public
POST	`/api/v1/auth/refresh`	Rotate refresh token; issue new access + refresh token pair	Public
GET	`/api/v1/auth/jwks`	Publish current RS256 public signing key(s) for verification/rotation (S0 §4.4)	Public
POST	`/api/v1/auth/verify`	Validate a bearer JWT; return `{valid, user{roles,permissions,name,employee_id,dept_scope}}`	Service Key
POST	`/api/v1/auth/logout`	Revoke all of the caller's active refresh tokens	JWT
GET	`/api/v1/auth/me`	Return the authenticated user + roles + permissions	JWT
POST	`/api/v1/auth/change-password`	Change own password (policy-enforced, Section 5.4)	JWT
GET	`/api/v1/users`	List users, paginated & searchable (D6)	JWT + `S1.identity.users.read`
POST	`/api/v1/users`	Create a user account directly (manual/service accounts — most accounts are auto-provisioned, Section 8)	JWT + `S1.identity.users.create`
GET	`/api/v1/users/{id}`	Show one user with roles & permissions	JWT + `S1.identity.users.read`
PUT	`/api/v1/users/{id}`	Update username/email/ employee_id/display_name	JWT + `S1.identity.users.update`

Method	Endpoint	Description	Auth
DELETE	`/api/v1/users/{id}`	Soft-delete a user (D13)	JWT + `S1.identity.users.delete`
POST	`/api/v1/users/{id}/deactivate`	Set `is_active=0`, revoke all refresh tokens	JWT + `S1.identity.users.deactivate`
POST	`/api/v1/users/{id}/force-logout`	Revoke **all** active refresh tokens for the user	JWT + `S1.identity.users.force_logout`
POST	`/api/v1/users/{id}/reset-password`	Admin-initiated password reset; sets `must_change_password=1`	JWT + `S1.identity.users.reset_password`
POST	`/api/v1/users/{id}/roles`	Assign a role (optionally scoped to `department_id`)	JWT + `S1.identity.users.assign_role`
DELETE	`/api/v1/users/{id}/roles/{rid}`	Remove a role assignment	JWT + `S1.identity.users.assign_role`
GET	`/api/v1/roles`	List roles	JWT + `S1.identity.roles.read`
POST	`/api/v1/roles`	Create a custom (non-system) role	JWT + `S1.identity.roles.create`
GET	`/api/v1/roles/{id}`	Show one role with its permissions	JWT + `S1.identity.roles.read`
PUT	`/api/v1/roles/{id}`	Update a role's display name/description	JWT + `S1.identity.roles.update`
DELETE	`/api/v1/roles/{id}`	Delete a non-system role (`is_system=0` only)	JWT + `S1.identity.roles.delete`
POST	`/api/v1/roles/{id}/permissions`	Replace (sync) a role's permission set; emits `permission.changed`	JWT + `S1.identity.roles.sync_permissions`
GET	`/api/v1/permissions`	List all permissions across all systems	JWT + `S1.identity.permissions.read`
GET	`/api/v1/permissions/{domain}`	List permissions filtered by domain, e.g. `identity`, `payroll`	JWT + `S1.identity.permissions.read`
GET	`/api/v1/audit-logs`	Query the audit log with filters (D6 pagination)	JWT + `S1.identity.audit_logs.read`
GET			

Method	Endpoint	Description	Auth
	`/api/v1/audit-logs/user/{id}`	Audit log entries for one user	JWT + `S1.identity.audit_logs.read`

CONVENTION

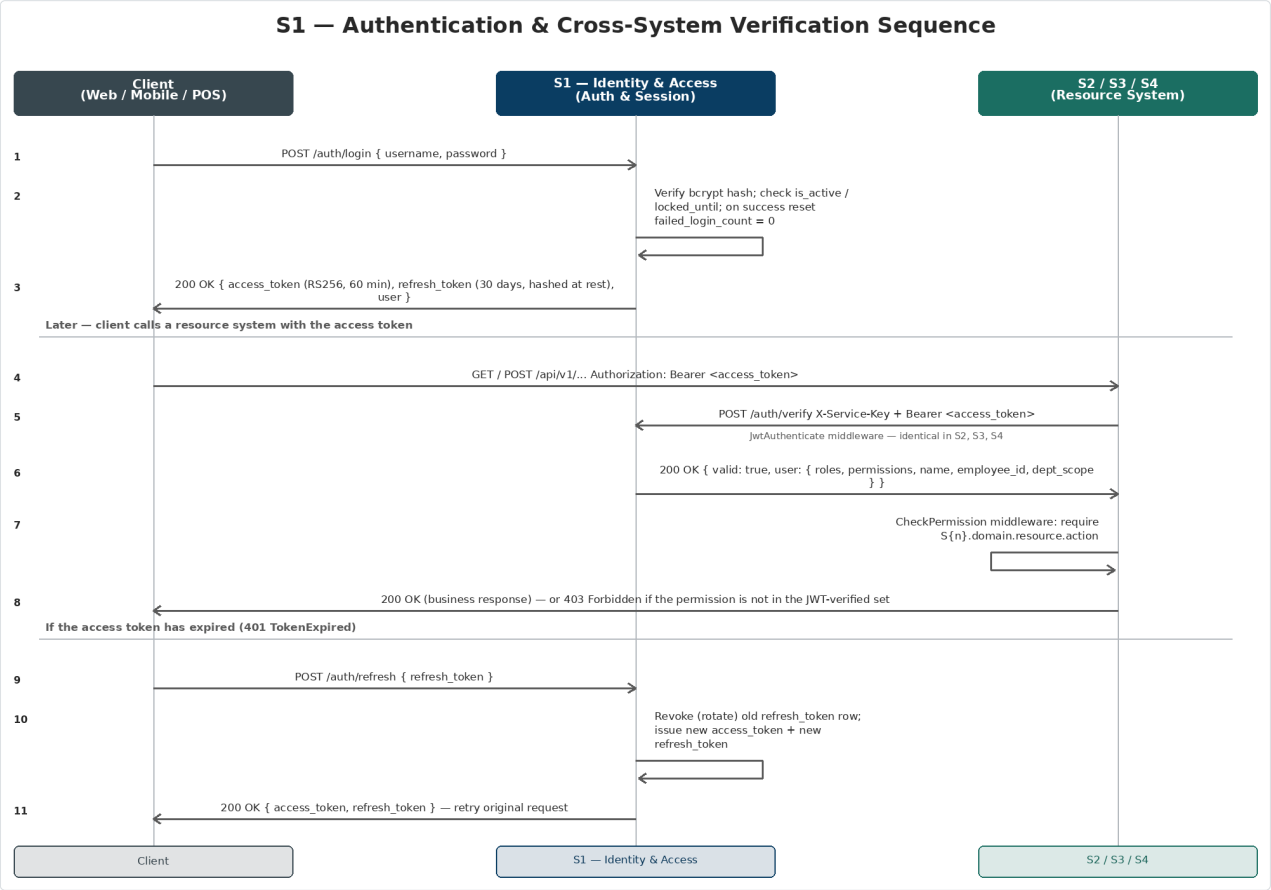
Pagination, error format, and rate limits. Every `GET` collection endpoint follows D6 (`?page=&per_page=` , default 25/max 100). Errors follow the D7 envelope (`{"error":{"code","message","details"}}`). `/auth/login` and `/auth/refresh` are gateway-rate-limited per S0 §3 (10/min and 30/min per IP respectively), independent of and in addition to S1's own account-lockout policy (Section 5.4).

---

5. Authentication & Session Management

5.1 End-to-end sequence

The diagram below covers the full lifecycle of a request's credentials: initial login, the cross-system verification call every other system makes, and the refresh flow when the access token expires.



Eleven-step sequence: (1-3) login issues an access token and refresh token; (4-8) a resource system (S2/S3/S4) verifies the bearer token with S1 and enforces RBAC; (9-11) the client refreshes an expired access token, with S1 rotating the refresh token.

5.2 JWT payload

S1 signs the following payload with its RS256 private key (`exp` = `iat` + 3600 seconds):

```
{
  "sub": "42",
  "employee_id": "17",
  "username": "abel.tadesse",
  "name": "Abel Tadesse",
  "roles": ["hr_manager", "report_viewer"],
  "permissions": ["S2.hr.employees.read", "S2.payroll.runs.read"],
  "dept_scope": "3",
  "iat": 1750000000,
  "exp": 1750003600,
```

```
  "iss": "wonderland-identity"
}
```

CROSS-REFERENCE

This is the same payload structure defined in S0 §4.2. `name` is `users.display_name` (ID2); `dept_scope` is the `department_id` from the user's `user_roles` row, if any. `roles` and `permissions` are computed from `user_roles` → `roles` → `role_permissions` → `permissions` at token-issue time and are **not** re-checked against the database on every request — S2–S4 trust the verified JWT contents for the lifetime of that single request (per-request only, never persisted, per S0 §4.1 step 5).

5.3 Token issuance & rotation rules

Rule	Detail
Access token TTL	60 minutes (`JWT_TTL=60`)
Refresh token TTL	30 days (`JWT_REFRESH_TTL=43200` minutes)
Refresh token storage	SHA-256 hash only (`refresh_tokens.token_hash`); the raw value is never persisted
Rotation on refresh	`POST /auth/refresh` revokes (`revoked_at=now()`) the presented refresh token and issues a brand-new access + refresh pair — refresh tokens are single-use
Logout	`POST /auth/logout` revokes **all** of the caller's active refresh tokens
Force-logout	`POST /users/{id}/force-logout` (admin action) revokes all of a *target* user's refresh tokens immediately
Signing algorithm	RS256. Private key never leaves S1; public key(s) published at `GET /auth/jwks` with `kid` for rotation (S0 §4.4)

5.4 Password policy & account lockout

Setting	Value	`.env` key
Minimum length	10 characters	`PASSWORD_MIN_LENGTH=10`
Complexity	Must include uppercase, a number, and a symbol	`PASSWORD_REQUIRE_UPPER=true`
Expiry	90 days (`password_changed_at` checked at login; if expired, `must_change_password` is set)	`PASSWORD_EXPIRY_DAYS=90`
History	Last 5 password hashes retained; reuse rejected by `PasswordPolicyService`	`PASSWORD_HISTORY=5`
Lockout threshold	5 consecutive failed logins	`MAX_FAILED_LOGINS=5`
Lockout duration	30 minutes (`locked_until = now() + 30 min`)	`LOCKOUT_MINUTES=30`

`failed_login_count` increments on each `login.failed` audit event and resets to `0` on `login.success`. While `locked_until` is in the future, `/auth/login` returns `403` regardless of credential correctness (and does **not** reset the counter, to avoid timing-based account enumeration).

5.5 Service-to-service authentication

`POST /api/v1/auth/verify` is the **only** S1 endpoint guarded by `ServiceKeyAuthenticate` rather than a user JWT (S0 §4.3). It accepts `INTERNAL_KEY_CURRENT` / `INTERNAL_KEY_PREVIOUS` for rotation (D8). Requests without a valid `X-Service-Key` header receive `401 INVALID_SERVICE_KEY` regardless of the bearer token's validity — this prevents external actors from using `/auth/verify` as a JWT-introspection oracle.

---

6. RBAC — Roles & Permission Catalog

6.1 S1's permission catalog (`domain = identity`)

These are the permission strings S1 itself defines and checks via its `CheckPermission` middleware. Naming follows ID9: `S1.identity.{resource}.{action}`.

Permission String	Guards	Default Role(s)
`S1.identity.users.read`	`GET /users`, `GET /users/{id}`	super_admin, general_manager, report_viewer
`S1.identity.users.create`	`POST /users`	super_admin
`S1.identity.users.update`	`PUT /users/{id}`	super_admin
`S1.identity.users.delete`	`DELETE /users/{id}`	super_admin
`S1.identity.users.deactivate`	`POST /users/{id}/deactivate`	super_admin
`S1.identity.users.force_logout`	`POST /users/{id}/force-logout`	super_admin
`S1.identity.users.reset_password`	`POST /users/{id}/reset-password`	super_admin
`S1.identity.users.assign_role`	`POST /users/{id}/roles`, `DELETE /users/{id}/roles/{rid}`	super_admin
`S1.identity.roles.read`	`GET /roles`, `GET /roles/{id}`	super_admin, general_manager, report_viewer
`S1.identity.roles.create`	`POST /roles`	super_admin
`S1.identity.roles.update`	`PUT /roles/{id}`	super_admin
`S1.identity.roles.delete`	`DELETE /roles/{id}`	super_admin
`S1.identity.roles.sync_permissions`	`POST /roles/{id}/permissions`	super_admin
`S1.identity.permissions.read`	`GET /permissions`, `GET /permissions/{domain}`	super_admin, general_manager, report_viewer
`S1.identity.audit_logs.read`	`GET /audit-logs`, `GET /audit-logs/user/{id}`	super_admin, general_manager, report_viewer

`(authenticated only)` — no specific permission required, any valid JWT suffices: `/auth/logout`, `/auth/me`, `/auth/change-password`.

> **DESIGN DECISION**
>
> **Change from the original Appendix A1 catalog.** The original M1 permission catalog granted `hr_manager` direct write access to `M1.users.*` (create/update/deactivate/reset-password) to support onboarding. Under the four-system design, **S2's employee lifecycle events automatically provision, sync, and deactivate the linked S1 account** (ID1, ID2, Section 8) — `hr_manager` no longer calls S1's user-management API directly. This matches the **System Access Matrix in S0 §7.2**, where `hr_manager` has **no direct S1 access** (–), while `general_manager` and `report_viewer` have **Read** access across `users`, `roles`, `permissions`, and `audit_logs` for security/compliance visibility. `S1.identity.users.create` remains available to `super_admin` for the rare manual/service-account case.

6.2 System roles (seeded, `is_system=1`)

All 12 roles from the platform-wide System Access Matrix (S0 §7.2) are seeded into S1's `roles` table — S1 is the single source of role definitions for every system. `is_system=1` roles cannot be deleted (`RoleController::destroy` returns `403`).

Role Slug	Display Name	S1 Access (this system)
`super_admin`	Super Administrator	Full — all `S1.identity.*` permissions
`general_manager`	General Manager	Read — `users`, `roles`, `permissions`, `audit_logs`
`finance_manager`	Finance Manager	— (no S1 permissions)
`accountant`	Accountant	—
`hr_manager`	HR Manager	— (see decision box above)
`payroll_officer`	Payroll Officer	—
`department_head`	Department Head	—
`inventory_manager`	Inventory Manager	—
`restaurant_manager`	Restaurant Manager	—
`receptionist`	Receptionist	—
`cashier`	Cashier	—
`report_viewer`	Report Viewer	Read — `users`, `roles`, `permissions`, `audit_logs`

> **CROSS-REFERENCE**
> This table only documents each role's **S1** permissions. A role's permissions for S2/S3/S4 domains are defined in those systems' SDDs but are stored as `role_permissions` rows in this system's database (Section 3.3 note). The full cross-system picture is S0 §7.2's Role → System Access Matrix.

6.3 `CheckPermission` middleware contract

Every protected route (other than the `(authenticated only)` set) is decorated with `middleware('permission:S1.identity.<resource>.<action>')`. The middleware:

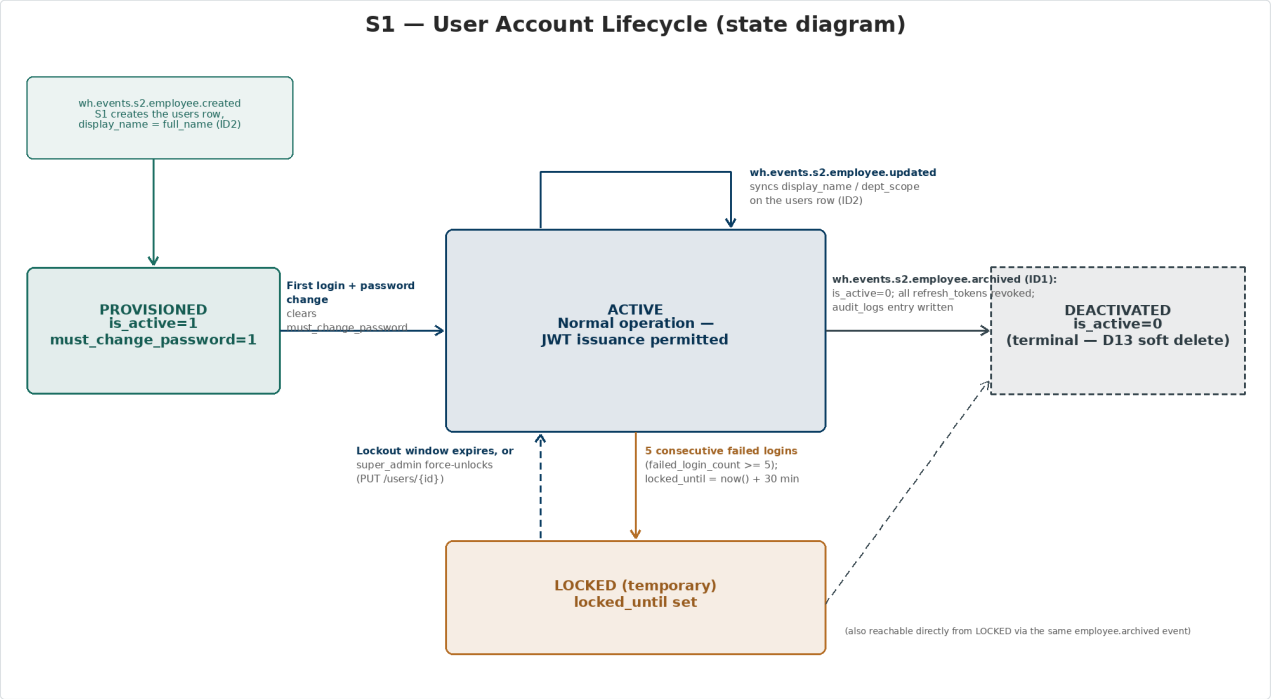
1. Reads `permissions` from the request's locally-decoded JWT payload (Section 2.2) — a flat array of permission strings.
2. Returns `403 FORBIDDEN` (D7) if the route's required permission string is not present.
3. Performs **no database query** — the permission set was computed at token-issue time (Section 5.2) and is trusted for the request's lifetime.

6.4 Seeding the global `permissions` table

At deploy time, S1's database seeder aggregates the permission catalog from **all four** system SDDs (this document's Section 6.1, plus S2/S3/S4's equivalent sections) into the single `permissions` table, then applies each role's default grants into `role_permissions`. Adding a new permission to any system requires a migration in **S1's** database, not the system that owns the permission's domain — coordinate cross-team changes via the shared `permissions` seeder file.

7. Account Lifecycle

A `users` row moves through a small set of states, driven partly by the user/admin (login, password change, lockout) and partly by **events from S2** (ID1, ID2).



Four states — PROVISIONED, ACTIVE, LOCKED (temporary), and DEACTIVATED (terminal) — with the transitions between them, including the two S2 employee-lifecycle events that drive provisioning, profile sync, and deactivation.

State	Meaning	`users` columns
PROVISIONED	Account created automatically from `wh.events.s2.employee.created`; not yet used	`is_active=1`, `must_change_password=1`
ACTIVE	Normal operation; JWTs may be issued	`is_active=1`, `must_change_password=0`, `locked_until=NULL`
LOCKED (temporary)	Too many failed logins; logins rejected until `locked_until` passes	`is_active=1`, `locked_until` in the future
DEACTIVATED (terminal)	Linked employee record archived in S2, or admin deactivation	`is_active=0`; all `refresh_tokens` revoked

CONVENTION

Per D13, **DEACTIVATED is reached by setting `is_active=0`, never by deleting the row.** The state diagram shows a dashed “admin reactivation” path back to ACTIVE (`PUT /users/{id}` with `is_active=true`) for the rare case of a data-entry correction — this is a manual `super_admin` action, not part of any automated workflow, since S2 does not currently emit an `employee.reactivated` event.

8. Event-Driven Integration

S1 is one of the **three systems that run a `redis-bus-subscriber` Horizon worker** (S1, S2, S3 — ID8) and one of the **three systems with an `event_outbox` table** (S1, S2, S3 — D2, ID10).

8.1 Consumed events (from S2)

Channel	Listener	Payload	Effect on `users`
`wh.events.s2.employee.created`	`CreateLinkedUserAccount`	`employee_id, employee_number, full_name, department_id, position_id, pension_category, basic_salary, default_role`	**Insert** a new `users` row: `employee_id`, `display_name=full_name`, `username` derived from `full_name` (e.g. `first.last`, de-duplicated with a numeric suffix if needed), a generated temporary password, `must_change_password=1`. Insert a `user_roles` row for `default_role`, scoped to `department_id` if the role is department-scoped. Writes `audit_logs` event `user.provisioned`.
`wh.events.s2.employee.updated`	`SyncLinkedUserAccount`	`employee_id, full_name, department_id, position_id`	**Update** the matching `users` row: `display_name=full_name` (so the JWT `name` claim, ID2, stays accurate) and the `user_roles.department_id` scope if the employee's department changed.
`wh.events.s2.employee.archived`	`DeactivateLinkedUserAccount`	`employee_id, employment_status: "archived", archived_at, reason`	**Update** the matching `users` row: `is_active=0`. Revoke (`revoked_at=now()`) every row in `refresh_tokens` for that user. Write `audit_logs` event `user.deactivated` with `payload.reason` from the event. This is the **new behaviour introduced by ID1** — in the original 9-module design, an offboarded employee's S1 login remained active indefinitely.

Each listener is idempotent on `employee_id`: if the matching `users` row does not exist (e.g. the event is replayed, or arrives before the initial provisioning completed), `CreateLinkedUserAccount` / `SyncLinkedUserAccount` create-or-update by `employee_id`; `DeactivateLinkedUserAccount` is a no-op if the user is already `is_active=0`.

8.2 Emitted events

Channel	Trigger	Payload	Consumed By
`wh.events.s1.permission.changed`	`RoleController::syncPermissions()` or `UserController::assignRole()` / `removeRole()`	`{role_id, permission_ids: [...], action: "sync"|"grant"|"revoke"}`	S4 (invalidates its RBAC/ admin dashboard cache, S0 §5.2)

8.3 Outbox mechanics

Every emitted event is written to `event_outbox` (Section 3.8) **inside the same database transaction** as the triggering change (e.g. the `role_permissions` sync and the `event_outbox` insert commit together). The `outbox:publish` job runs every 10 seconds:

1. Select rows where `status='pending'`, ordered by `created_at`.
2. For each, `PUBLISH` the envelope (D3: `{event, version, occurred_at, source_system: "S1", request_id, payload}`) to `wh-redis-bus`.
3. On success: `status='published'`, `published_at=now()`.
4. On failure: increment `attempts`; after exhausting the backoff schedule `[10s, 60s, 5m, 30m, 2h]` (D2), set `status='failed'` and trigger `NotificationService::send()` to `super_admin` (D4) — which itself becomes an `audit_logs` entry.

8.4 Bus subscriber

S1's single `redis-bus-subscriber` Horizon worker subscribes to `wh.events.s2.employee.*` and dispatches the corresponding local listener (Section 8.1). Subscriber job failures follow Horizon's default retry policy (3 attempts, backoff `[10s, 60s, 5m]`) before landing in `failed_jobs`, which also triggers the D4 `super_admin` notification.

9. Audit Logging & Compliance

The `audit_logs` table (Section 3.7) is the platform's security record. Every row is written by `AuditService::log($event, $userId, $ipAddress, $payload = null)`, called from the listeners below:

Event	Trigger	Listener
`login.success`	`AuthController::login()` on success	`WriteAuditLog`
`login.failed`	`AuthController::login()` on bad credentials (`user_id=NULL`)	`WriteAuditLog`
`logout`	`AuthController::logout()`	`WriteAuditLog`
`permission.changed`	`RoleController::syncPermissions()` , `UserController::assignRole()` / `removeRole()`	`WriteAuditLog` (and emits the bus event, Section 8.2)
`user.deactivated`	`UserController::deactivate()` (admin) or `DeactivateLinkedUserAccount` (ID1 event)	`WriteAuditLog`
`user.provisioned`	`CreateLinkedUserAccount` (ID2 event)	`WriteAuditLog`
`password.reset`	`UserController::resetPassword()` (admin-initiated)	`WriteAuditLog`
`password.changed`	`AuthController::changePassword()` (self-service)	`WriteAuditLog`
`dr.restore_drill`	Quarterly DR restore drill (D14, operational process, not an API call)	Manual `AuditService::log()` invocation per the runbook

`GET /api/v1/audit-logs` supports filtering by `event` , `user_id` , and a date range, with D6 pagination. `GET /api/v1/audit-logs/user/{id}` is a convenience filter equivalent to `?user_id={id}` .

10. Security, Secrets & Operational Conventions

This section applies the platform-wide conventions (S0 §9, D1–D14) specifically to S1; it does not repeat their full definitions.

Convention	Application to S1
D2 Event outbox	S1 runs it (Section 8.3)
D3 Event envelope	S1's `permission.changed` events use the standard envelope with `source_system: "S1"`
D4 Retry / dead letter	S1's outbox and subscriber failures notify `super_admin` (also logged to `audit_logs`, Section 9)
D8 Secrets & rotation	S1 is where **JWT signing-key rotation** happens (`GET /auth/jwks` , S0 §4.4) — no other system is involved. S1 also participates in `INTERNAL_KEY_CURRENT` / `_PREVIOUS` rotation for its own `/auth/verify` endpoint like every system
D9 Logging & correlation	S1's logs include `X-Request-Id` from the gateway; every `audit_logs` row should additionally capture the originating `X-Request-Id` in `payload` where available
D12 Testing	S1 is a required half of every cross-system event-integration test pair involving S2 (employee provisioning/sync/deactivation) and is the callee in the S2↔S1, S3↔S1, S4↔S1 `/auth/verify` contract tests
D13 Soft delete	`users.deleted_at` (Laravel `SoftDeletes`), `users.is_active`, `audit_logs` append-only — no hard deletes anywhere in `wh_s1_db`
D14 Backup / DR	`wh_s1_db` is one of the 4 databases in the nightly encrypted `mysqldump` ; quarterly DR restore-drill results are logged to **this system's** `audit_logs` (Section 9) regardless of which database was restored

10.1 `.env` reference

```
APP_NAME=WH-S1-Identity-Access
APP_URL=http://s1-identity:9001
DB_CONNECTION=mysql
DB_DATABASE=wh_s1_db
JWT_ALGO=RS256
JWT_PRIVATE_KEY_PATH=/run/secrets/jwt_private.pem
JWT_PUBLIC_KEY_PATH=/run/secrets/jwt_public.pem
JWT_TTL=60                  # access token, minutes
JWT_REFRESH_TTL=43200       # refresh token, minutes (30 days)
REDIS_HOST=wh-redis         # Horizon queue connection
REDIS_BUS_HOST=wh-redis-bus
INTERNAL_KEY_CURRENT_FILE=/run/secrets/internal_key_current
INTERNAL_KEY_PREVIOUS_FILE=/run/secrets/internal_key_previous
PASSWORD_MIN_LENGTH=10
PASSWORD_REQUIRE_UPPER=true
PASSWORD_EXPIRY_DAYS=90
PASSWORD_HISTORY=5
```

```
MAX_FAILED_LOGINS=5
LOCKOUT_MINUTES=30
```

CROSS-REFERENCE

For the gateway routing block, the full Docker Compose topology, and the shared `wh-redis-bus` / `wh-mysql` services, see S0 Section 10 — Deployment Topology.

---

11. Non-Functional Requirements & Testing

Category	Requirement
Availability	S1 is a hard dependency for *every* request to S2–S4 (S0 §4.1, step 4). The gateway returns `503` (D7) if S1's health check fails — treat S1 outages as platform-wide outages
Latency	`/auth/verify` is called on every authenticated request platform-wide; target p99 < 50ms (in-process JWT decode + a single indexed `users` lookup, no joins beyond `roles` / `permissions` which are denormalized into the JWT at issue time)
Throughput	Size `/auth/verify` for the combined request volume of S2+S3+S4, not S1's own traffic
Security	RS256 only (no symmetric JWT signing); bcrypt for passwords; `audit_logs` append-only; service-key rotation per D8
Testing — unit/feature	PHPUnit + PHPStan against MySQL 8 + Redis 7 (D11). Cover: login success/failure/lockout, token rotation, password policy edge cases, RBAC middleware, soft-delete behaviour
Testing — contract	`GET /api/v1/openapi.json` validated against the calls in S0 §5.3 (every system calling `/auth/verify`)
Testing — event-integration	`docker-compose.test.yml` pairs per D12: **S2↔S1** (employee created/updated/archived → user provisioned/ synced/deactivated), **S1↔S4** (permission.changed → cache invalidation)
Testing — UAT	Tracked in S4's RTM/UAT tracker against the original M1 FR codes (Appendix B traceability, S0 §12)

12. Integration Contract Summary

This section is a developer-facing checklist — the complete set of promises S1 makes to, and depends on from, the other three systems. Every row traces back to a section in this document or in S0.

12.1 What S1 provides to S2 / S3 / S4

Contract	Detail
`POST /api/v1/auth/verify`	Section 4, Section 5.5 — must remain available and fast; this is the platform's single point of failure for authentication
JWT payload shape	Section 5.2 — `name`, `permissions`, `dept_scope` must always be populated and current
`GET /api/v1/auth/jwks`	Section 5.3 — used only if a resource system ever needs to verify locally (not currently exercised, but published per S0 §4.4)
`GET /api/v1/users`, `/roles`, `/audit-logs`	Section 4 — read-only, called by S4 for admin/security dashboards (S0 §5.3, §5.4); cached by S4 with TTLs from S0 §5.4
`wh.events.s1.permission.changed`	Section 8.2 — must be emitted (via outbox) on every `role_permissions` sync or role (re)assignment

12.2 What S1 depends on from S2

Dependency	Detail
`wh.events.s2.employee.created`	Section 8.1 — required for automated account provisioning (ID2). Must include `full_name`, `department_id`, `position_id`, `default_role`
`wh.events.s2.employee.updated`	Section 8.1 — required to keep `display_name` / `dept_scope` accurate (ID2)
`wh.events.s2.employee.archived`	Section 8.1 — required for automatic deactivation (ID1) — **this is a new dependency** that did not exist in the original M1 design; S2's SDD must confirm this event is emitted on every offboarding

12.3 Build-order implication

Because account provisioning depends on S2's `employee.created` event, **S1 and S2 should be developed with the event contract (Section 8.1 payload shape) agreed first**, even though the two systems are built by separate teams. S1 can and should ship with a seeded `super_admin` account and at least one seeded user per system role for the other three teams to develop against, independent of the S2 event integration being complete.

End of Document — S1, Identity & Access Platform, v1.0